

LEMBAR KERJA PRAKTIKUM

PRAKTIKUM 3

Evaluasi Sistem Temu Kembali

Mata Kuliah Sistem Temu Kembali Informasi · Pertemuan 3 · Semester 5

Program Studi	Teknik Komputer
Perangkat	VS Code (Extension Jupyter Notebook)
Nama Mahasiswa	Ahmad Jalil Dzaky S.
NIM / Kelas	240210502021 / Sistem Temu Kembali B
Tanggal Praktikum	3 September 2026

A. Daftar 10 judul Skripsi (Tugas Akhir) Mahasiswa Teknik Komputer dari Tugas 2

1. Analisis Taktikal Futsal Menggunakan Chatbot Ai Berbasis Large Language Model
2. PENGEMBANGAN MODUL RETRIEVAL BERBASIS VECTOR SEARCH DENGAN MEKANISME FALLBACK WEB SEARCH DAN MODUL TOOL CALLING PADA ARSITEKTUR AGENTIC AI CHATBOT UNTUK INFORMASI IKLIM
3. PENGEMBANGAN SISTEM PENGANALISIS DATA PORTOFOLIO PERKULIAHAN DENGAN ARSITEKTUR AGENTIC AI
4. VISUALISASI DATA DARI BERBAGAI SISTEM JARINGAN SOSIAL SEBAGAI DISCRETE MOVING POINTS
5. Pengembangan Platform Marketplace Tiket Konser Berbasis Smart Contract Pada Jaringan Layer-2 Ethereum
6. PENGEMBANGAN KONTROL AKSES DALAM KERANGKA KERJA SISTEM MANAJEMEN REKAM MEDIS ELEKTRONIK BERBASIS IOTA
7. Alat Pengisi Token Listrik Menggunakan Sistem 2-Axis Gantry Berbasis Iot
8. Putting Responsible AI To Work With AI Governance As Data & AI Patner Technical Sp
9. AI Chatbot Development for Computer Science Curriculum Based Academic Consultation using Retrieval Augmented Generation (RAG)
10. SISTEM MONITORING RUMAH KACA BUDIDAYA PEMBIBITAN TANAMAN NILAM BERBASIS INTERNET OF THINGS DAN DATABASE REALTIME

B. Kode Program dan Output

Kode Program Praktikum :

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

judul_lagu = [
    "Monokrom",
    "Hati-Hati di Jalan",
    "Pamit",
    "Sepatu",
    "Diri"
]

daftar_lirik = [
    """
    kenangan masa lalu tersimpan dalam ingatan
    keluarga sahabat dan kasih sayang
    cerita lama tetap hidup dalam kenangan
    terima kasih untuk semua yang pernah ada
    """,
    """
    perjalanan bersama berakhir dengan perpisahan
    kita memilih jalan yang berbeda
    semoga kamu menemukan bahagia
    walaupun tidak lagi berjalan bersama
    """,
    """
    aku memilih pergi dari hubungan ini
    hati yang lelah membutuhkan waktu
    perpisahan bukan berarti tidak pernah cinta
    semoga kita menemukan bahagia masing-masing
    """,
    """
    kita seperti sepasang sepatu yang berjalan bersama
    namun memiliki arah yang berbeda
    ada rasa cinta tetapi sulit untuk bersama
    akhirnya kita harus menerima perpisahan
    """,
    """
    jangan terlalu keras kepada dirimu sendiri
    kamu berhak mendapatkan bahagia
    belajar mencintai diri sendiri
    menerima semua kekurangan dan kelebihan
    """,
]

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(daftar_lirik)

query = [
    "cinta",
    "patah",
    "hati"
]

skor = cosine_similarity(
    vectorizer.transform(query),
    X
)

relevan = {
    "cinta": {1, 3, 4},
    "patah": {2, 3, 4},
    "hati": {1, 2, 3, 5}
}
```

```

def precision_at_k(hasil, relevan, k):
    return sum(
        1 for d in hasil[:k]
        if d in relevan
    ) / k

def recall(hasil, relevan):
    cocok = sum(
        1 for d in hasil
        if d in relevan
    )

    return cocok / len(relevan)

def f1(p, r):
    return 0.0 if p + r == 0 else 2 * p * r / (p + r)

def average_precision(hasil, relevan):
    hit = 0
    total = 0.0

    for i, d in enumerate(hasil, start=1):
        if d in relevan:
            hit += 1
            total += hit / i

    return total / len(relevan)

```

```

ap_semua = []

for i, q in enumerate(query):
    hasil_skor = []

    for doc_id, nilai_skor in enumerate(skor[i]):
        hasil_skor.append(
            (doc_id + 1, nilai_skor)
        )

    hasil_skor.sort(
        key=lambda x: x[1],
        reverse=True
    )

    ranking = [
        doc_id
        for doc_id, nilai_skor in hasil_skor
    ]

    p = precision_at_k(
        ranking,
        relevan[q],
        5
    )

    r = recall(
        ranking,
        relevan[q]
    )

```

```

f = f1(p, r)

ap = average_precision(
    ranking,
    relevan[q]
)

ap_semua.append(ap)

print("=" * 60)
print(f"KUERI {i + 1} : {q}")
print(f"Ground Truth : {relevan[q]}")
print(f"Hasil      : {ranking}")

print(f"Precision@5 : {p:.4f}")
print(f"Recall      : {r:.4f}")
print(f"F1 Score    : {f:.4f}")
print(f"AP         : {ap:.4f}")

MAP = sum(ap_semua) / len(ap_semua)
print(f"MAP : {MAP:.4f}")

```

Output :

```

=====
KUERI 1 : cinta
Ground Truth : {1, 3, 4}
Hasil      : [3, 4, 1, 2, 5]
Precision@5 : 0.6000
Recall      : 1.0000
F1 Score    : 0.7500
AP         : 1.0000
=====
KUERI 2 : patah
Ground Truth : {2, 3, 4}
Hasil      : [1, 2, 3, 4, 5]
Precision@5 : 0.6000
Recall      : 1.0000
F1 Score    : 0.7500
AP         : 0.6389
=====
KUERI 3 : hati
Ground Truth : {1, 2, 3, 5}
Hasil      : [3, 1, 2, 4, 5]
Precision@5 : 0.8000
Recall      : 1.0000
F1 Score    : 0.8889
AP         : 0.9500
MAP : 0.8630

```

Kode Program Tugas :

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Dataset 10 judul skripsi
judul_skripsi = [
    "Analisis Taktikal Futsal Menggunakan Chatbot Ai Berbasis Large Language Model",
    "PENGEMBANGAN MODUL RETRIEVAL BERBASIS VECTOR SEARCH DENGAN MEKANISME FALLBACK WEB SEARCH DAN MODUL TOOL CALLING PADA ARSITEKTUR AGENTIC AI CHATBOT UNTUK INFORMASI IKLIM",
    "PENGEMBANGAN SISTEM PENGANALISIS DATA PORTOFOLIO PERKULIAHAN DENGAN ARSITEKTUR AGENTIC AI",
    "VISUALISASI DATA DARI BERBAGAI SISTEM JARINGAN SOSIAL SEBAGAI DISCRETE MOVING POINTS",
    "Pengembangan Platform Marketplace Tiket Konser Berbasis Smart Contract Pada Jaringan Layer-2 Ethereum",
    "PENGEMBANGAN KONTROL AKSES DALAM KERANGKA KERJA SISTEM MANAJEMEN REKAM MEDIS ELEKTRONIK BERBASIS IOTA",
    "Alat Pengisi Token Listrik Menggunakan Sistem 2-Axis Gantry Berbasis Iot",
    "Putting Responsible AI To Work With AI Governance As Data & AI Patner Technical Sp",
    "AI Chatbot Development for Computer Science Curriculum Based Academic Consultation using Retrieval Augmented Generation (RAG)",
    "SISTEM MONITORING RUMAH KACA BUDIDAYA PEMBIBITAN TANAMAN NILAM BERBASIS INTERNET OF THINGS DAN DATABASE REALTIME"
]

# TF-IDF
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(judul_skripsi)

# Query
query = ["AI", "Jaringan", "IoT"]

# Cosine similarity
skor = cosine_similarity(
    vectorizer.transform(query),
    X
)
```

```
relevan = {
    "AI": {1, 2, 3, 8, 9},
    "Jaringan": {4, 5},
    "IoT": {7, 10}
}

# Fungsi Precision@K
def precision_at_k(hasil, relevan, k):
    return sum(1 for d in hasil[:k] if d in relevan) / k

# Fungsi Recall
def recall(hasil, relevan):
    cocok = sum(1 for d in hasil if d in relevan)
    return cocok / len(relevan)

# Fungsi F1
def f1(p, r):
    return 0.0 if p + r == 0 else 2 * p * r / (p + r)
```

```

def average_precision(hasil, relevan):
    hit = 0
    total = 0.0

    for i, d in enumerate(hasil, start=1):
        if d in relevan:
            hit += 1
            total += hit / i

    return total / len(relevan)

# Menyimpan AP setiap query
ap_semua = []

# Proses setiap query
for i, q in enumerate(query):

    hasil = []

    for doc_id, nilai_skor in enumerate(skor[i]):
        hasil.append(
            (doc_id + 1, nilai_skor)
        )

    # Urutkan berdasarkan skor tertinggi
    hasil.sort(key=lambda x: x[1], reverse=True)

    # Ambil ID dokumen saja
    ranking = [doc_id for doc_id, nilai in hasil]

```

```

# Hitung metrik
p = precision_at_k(ranking, relevan[q], 3)
r = recall(ranking, relevan[q])
f = f1(p, r)
ap = average_precision(ranking, relevan[q])

ap_semua.append(ap)

print(f"Query: {q}")
print(f"Ranking: {ranking}")
print(f"Precision@3: {p:.4f}")
print(f"Recall: {r:.4f}")
print(f"F1: {f:.4f}")
print(f"AP: {ap:.4f}")
print()

# Mean Average Precision
map_score = sum(ap_semua) / len(ap_semua)

print(f"MAP: {map_score:.4f}")

```

Output

```

Query: AI
Ranking: [8, 3, 1, 9, 2, 4, 5, 6, 7, 10]
Precision@3: 1.0000
Recall: 1.0000
F1: 1.0000
AP: 1.0000

Query: Jaringan
Ranking: [4, 5, 1, 2, 3, 6, 7, 8, 9, 10]
Precision@3: 0.6667
Recall: 1.0000
F1: 0.8000
AP: 1.0000

Query: IoT
Ranking: [7, 1, 2, 3, 4, 5, 6, 8, 9, 10]
Precision@3: 0.3333
Recall: 1.0000
F1: 0.5000
AP: 0.6000

MAP: 0.8667

```

C. Analisis Output

Query IoT menunjukkan performa pencarian yang paling rendah dibandingkan dengan query AI dan Jaringan. Hal ini dapat dilihat dari nilai Precision@3 sebesar 0.3333, F1 sebesar 0.5000, serta AP sebesar 0.6000, yang menjadi nilai terendah di antara ketiga query. Walaupun nilai Recall IoT mencapai 1.0000, dokumen relevan lainnya, yaitu dokumen 10, baru muncul pada posisi ke-10. Kondisi tersebut disebabkan oleh perbedaan istilah, yaitu dokumen 10 menggunakan kata "Internet of Things", sementara query menggunakan "IoT", sehingga TF-IDF tidak mampu mengidentifikasi keduanya sebagai istilah yang memiliki makna serupa. Oleh karena itu, hasil tersebut menunjukkan bahwa TF-IDF masih memiliki keterbatasan dalam menangkap hubungan semantik atau kesamaan makna antaristilah.